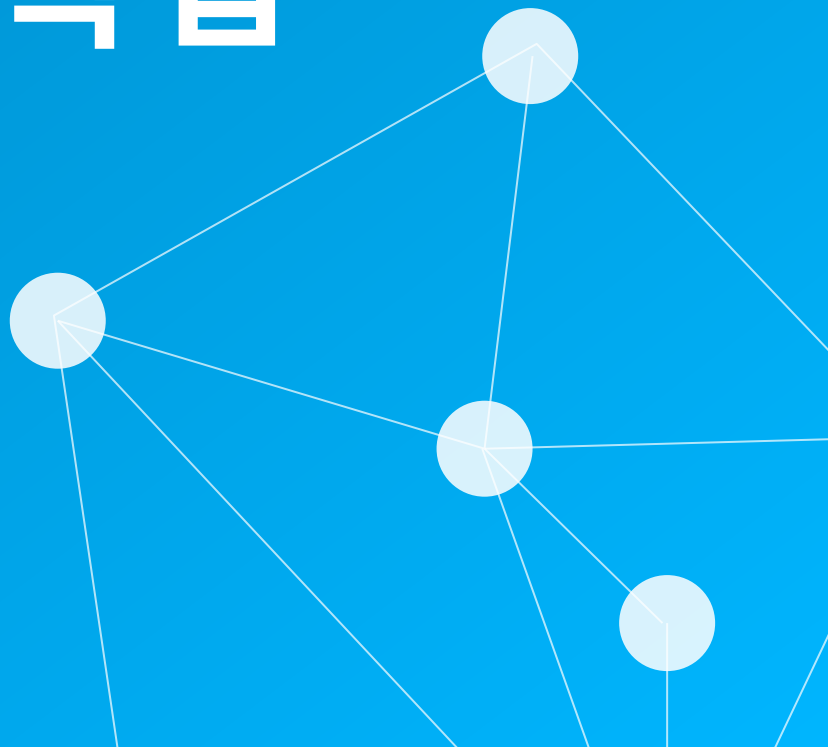
CHAPTER 07

동적 계획법

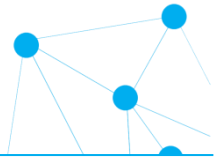


동적 계획법



1. 동적 계획법이란?
2. 피보나치수열
3. 0-1 배낭 채우기

1. 동적 계획법이란?



- 동적 프로그래밍(dynamic programming)
  - 1950년대 미국의 수학자인 벨맨(Richard Bellman)
  - 다단계 의사 결정 프로세스를 최적화하는 일반적인 방법
    → 동적 계획법으로 번역

- 동적 계획법
  - 분할 정복 기법과 유사
  - 부분 문제들의 답을 저장했다가 다시 꺼내서 사용
    → 같은 부분 문제를 다시 풀지 않도록 하는 것이 핵심
  - 추가적인 메모리 필요 : 공간으로 시간을 버는 전략의 일종

동적 계획법을 이용한 문제 해결 전략

- 문제의 특성
  (1) 중복된 부분 문제(overlapping subproblem)
  (2) 최적 부분 구조(optimal substructure)
    - 부분 문제의 최적해들을 이용하면 전체 문제의 최적해를 구할 수 있는 구조

- 동적 계획법의 패러다임과 구현 방법

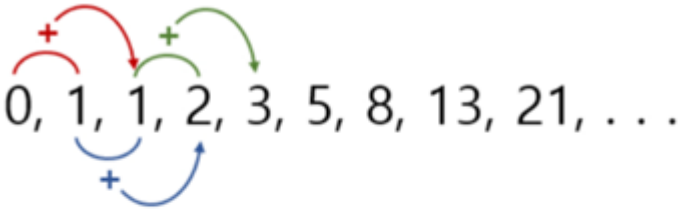


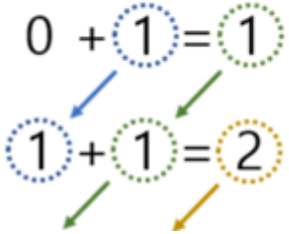


피보나치 수열

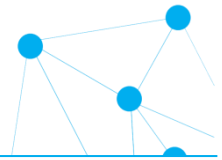
2. 피보나치 수열

- 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144…





피보나치 수열 알고리즘 (반복 구조)



```
def fib_iter(n):
    if n <= 1 : return n

    last = 0
    current = 1

    for i in range(1, n) :      # n-1번 반복
        tmp = current
        current += last         # 이전항목과 현재항목을 더함
        last = tmp              # 현재항목을 이전항목으로 함

    return current

# 테스트
n = 6
print(f"피보나치 {n}번째 수: {fib_iter(n)}")
```

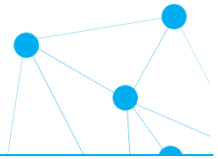
	last	current
	0	1
i=1	1	1
i=2	1	2
i=3	2	3
i=4	3	5
i=5	5	8

- $O(n)$

피보나치 6번째 수 : 8

0, 1, 1, 2, 3, 5, 8

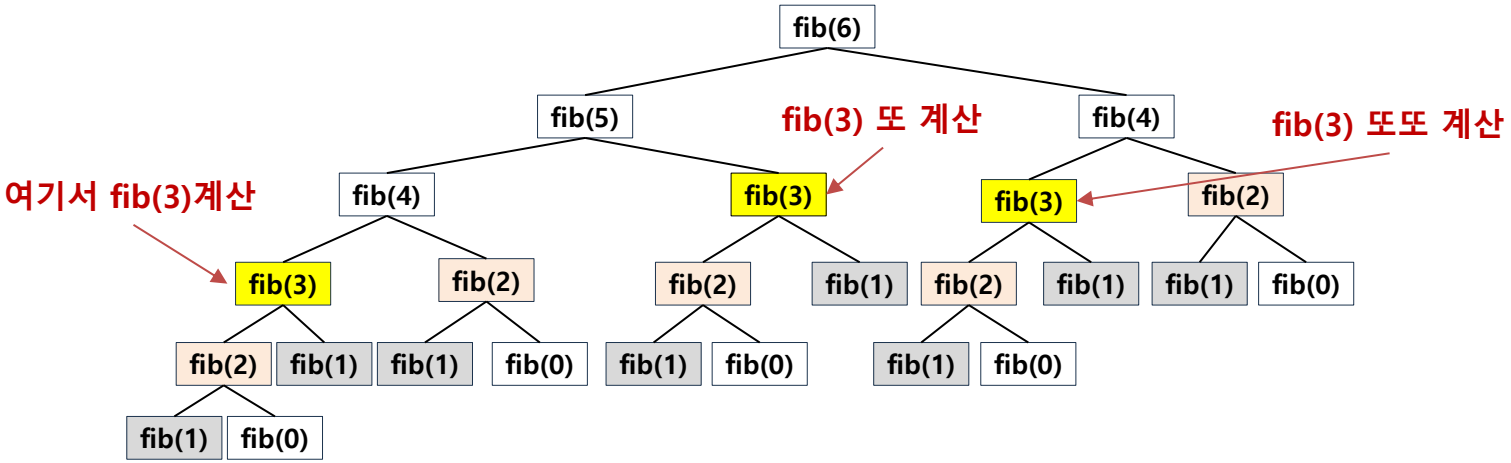
피보나치 수열 알고리즘 (순환 구조)



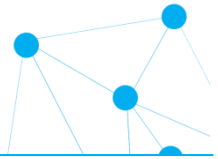
- 순환 관계식

$$fib(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ fib(n-1) + fib(n-2) & otherwise \end{cases}$$

  - **순환 구조(분할 정복)** : 트리구조의 전위순회
  - 심각한 중복 계산 문제 : ***O(2ⁿ)***
    - **fib(6)을 풀기위해 fib(1)을 8번 호출**
    - **fib(10)을 구하기 위해 100번 넘게 함수 호출**



피보나치 수열 알고리즘 (분할 정복)



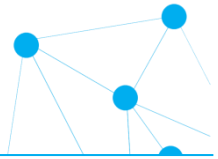
```
def fib(n):
    if n <= 1:  # Base Case
        return n

    return fib(n - 1) + fib(n - 2)

# 테스트
n = 6
print(f"피보나치 {n}번째 수: {fib(n)}")    # 피보나치 6번째 수 : 8
```

- $T(n) = T(n-1) + T(n-2) + C$
- $O(2^n)$: 아주 비효율적이 알고리즘
- 순환구조(분할정복)는 코드는 간결하나, 같은 계산을 반복하기 때문에 n이 큰 경우는 매우 비효율적

피보나치 수열 알고리즘 분석



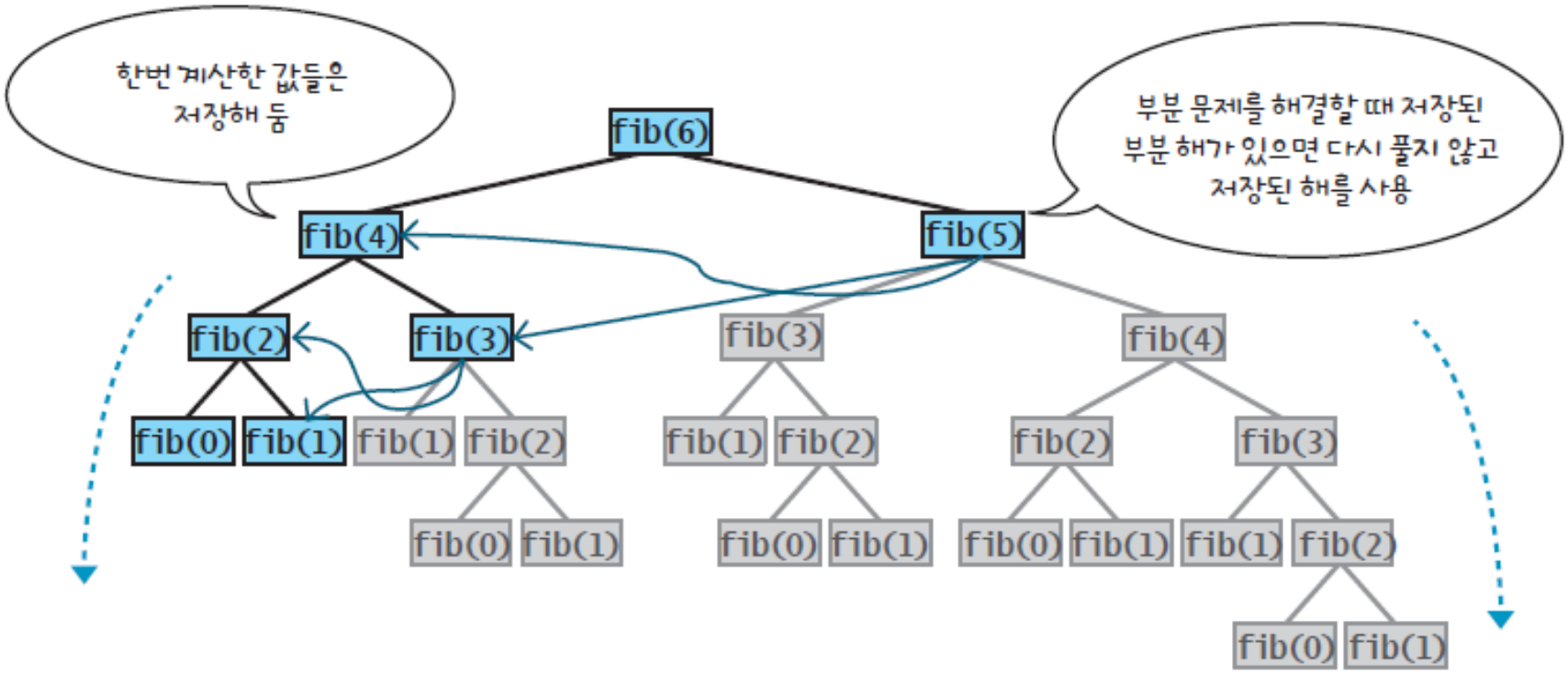
- **팩토리얼** 순환구조(분할정복) 알고리즘
  - 5! = 5 * 4!, 4! = 4 * 3! …
  - 단 한번도 같은 숫자의 팩토리얼 호출이 발생하지 않음
  - ***O(n)***

- **피보나치** 순환구조(분할정복) 알고리즘
  - 심각한 중복 계산 문제 발생 : $O(2^n)$

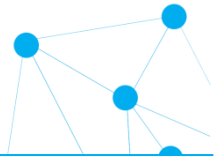
- **동적 계획법**
  - **한번 구한 답을 저장하여 재사용**
  - 구현방법
    - **메모이제이션(memoization)**: 하향식(top-down)
    - **테이블화(tabulation)**: 상향식(bottom-up)

메모이제이션을 이용한 피보나치 수열

- 메모이제이션(memoization)
  - 함수의 결과를 저장할 메모리를 미리 준비해서 한번 계산한 값을 저장해 두었다가 재활용하는 방법
  - 하향식(top-down) 접근 : 스택



피보나치 수열 (메모이제이션)



```
def fib_mem(n) :
    if mem[n] == None :              # 계산된 부분은 패스, 계산 안된 부분만 계산
        if n <= 1 : mem[n] = n  # Base Case
        else: mem[n] = fib_mem(n-2) + fib_mem(n-1)
        print(f'n={n} {mem}')
    return mem[n]

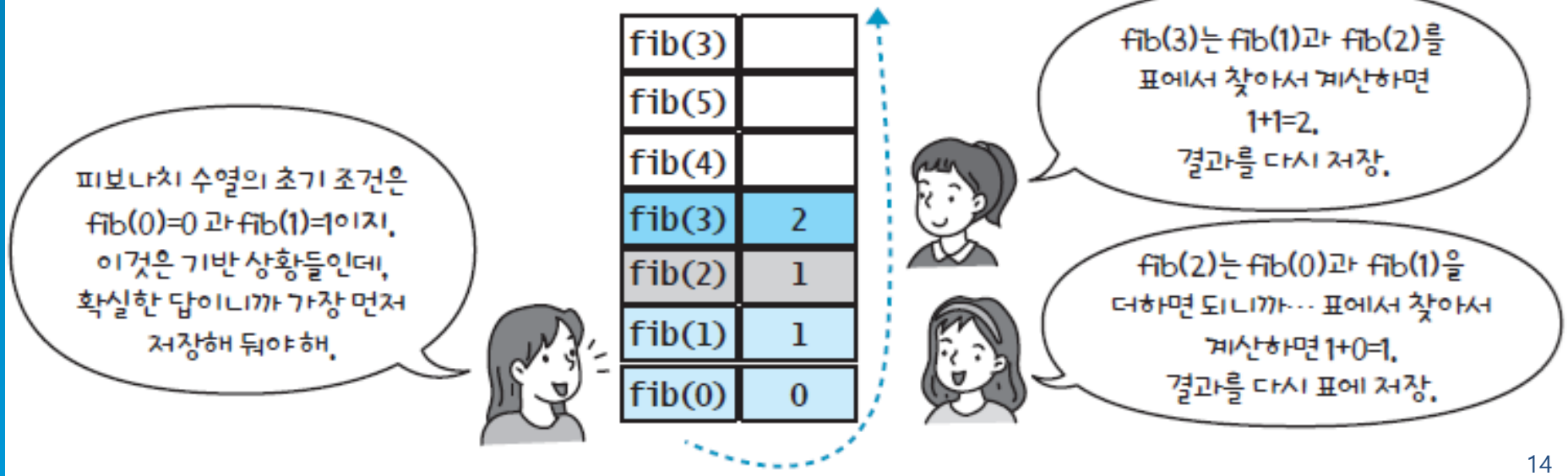
#테스트
n = 6
mem = [None] * (n+1)                 # mem[k]는 k번째 피보나치수열값
print(f'피보나치(메모이제이션): {fib_mem(n)}')
```

- 결과 저장을 위한 메모리 필요 : mem(n+1개)
  - mem[k]에는 k번째 피보나치수 저장
- 성능 : **mem[n]==None 부분만 계산**
  - 시간 복잡도: ***O(n)***
  - 공간 복잡도: ***O(n)***

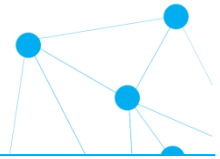
```
n=0 [0, None, None, None, None, None, None]
n=1 [0, 1, None, None, None, None, None]
n=2 [0, 1, 1, None, None, None, None]
n=3 [0, 1, 1, 2, None, None, None]
n=4 [0, 1, 1, 2, 3, None, None]
n=5 [0, 1, 1, 2, 3, 5, None]
n=6 [0, 1, 1, 2, 3, 5, 8]
피보나치(메모이제이션): 8
```

테이블화를 이용한 피보나치 수열

- 테이블화(tabulation, 타뷸레이션)
  - 테이블 항목들을 순서대로 채워나가는 것에 초점
  - 상향식(bottom-up) 접근 : fib(0) → fib(n)의 결과 저장
  - 계산은 Base Case부터 시작 : 답을 알고있는 fib(0), fib(1)의 결과를 테이블에 저장
  - 다음에는 테이블을 이용하여 계산하고 그 결과를 다시 테이블에 저장하는 과정을 반복
  - 배열이나 연결리스트 구조



피보나치 수열 (테이블화)



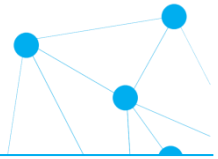
- 테이블화(tabulation, 타뷸레이션)
  - **알고리즘 : f[n] = f[n-2] + f[n-1]**
  - 1차원 테이블 : n+1
  - base case : fib(0)=0, fib(1)=1
  - n = 6

	0	1	2	3	4	5	6
n	None	None	None	None	None	None	None

	0	1	2	3	4	5	6
n	0	1	F(0)+F(1) = 1	F(1)+F(2) = 1 + 1	F(2)+F(3) = 1+ 2	F(3)+F(4) = 2 + 3	F(4)+F(5) = 3 + 5

  - 피보나치 6번째 수 : 8

피보나치 수열 (테이블화)



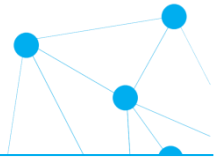
```
def fib_tab(n) :
    f = [None] * (n+1)  # 테이블을 만들고
    f[0] = 0            # Base Case
    f[1] = 1            # Base Case
    for i in range(2, n + 1):    # 상향식
        f[i] = f[i-2] + f[i-1]
    print(f'n={n} {f}')
    return f[n]
```

n=6 [0, 1, 1, 2, 3, 5, 8]
피보나치(테이블화) 6번째 수 : 8

```
#테스트
n = 6
print(f'피보나치(테이블화) {n}번째 수 : {fib_tab(n)}')
```

- 결과 저장을 위한 테이블 : f(n+1개)
  - f[k]에는 k번째 피보나치수 저장
- 성능
  - 시간 복잡도: *O(n)*
  - 공간 복잡도: *O(n)*

연습문제

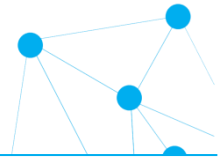


1. 피보나치 수열의 순환식 F(n) = F(n-1) + F(n-2)라고 할 때 base case는 무엇인가?
2. 동적계획법에서 큰 문제를 작은 하위 문제로 나누어 해결하는 과정 중 이미 계산된 값을 저장하여 다시 계산되지 않도록 하는 기법은 무엇인가?
3. 단순 재귀 호출로 피보나치 수열 알고리즘 작성시 시간 복잡도는?
4. 메모이제이션을 적용한 재귀 방식의 피보나치 알고리즘이 시간 복잡도는?
5. 상향식 접근 방법을 사용하는 동적계획법에서 사용되는 자료구조는?
6. 재귀 호출을 사용할 때 시스템 내부적으로 호출 정보를 저장하기 위해 사용되는 메모리 영역은?
7. 동적계획법 중 큰 문제를 작은 문제로 쪼개며 재귀적으로 해결하는 방식은?
8. 메모이제이션을 사용하지않은 재귀 알고리즘에서 F(5)를 구할 때 F(1)은 몇 번 반복되는가?
9. 피보나치 수열 알고리즘이 동적계획법의 대상이 되는 결정적인 이유는?



0-1 배낭 채우기

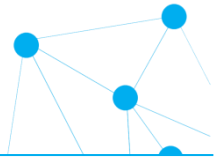
3. 배낭 채우기



- 0-1 배낭 채우기
  - 억지 기법 : 완전 탐색
    - 원소의 개수가 n인 집합의 부분집합의 수 2^n : $\boldsymbol{O(2^n)}$
  - 탐욕적 기법 : 최적해를 구하지 못함 (분할가능한 배낭채우기로 가능)
  - 분할정복 기법 : $\boldsymbol{O(2^n)}$ 비효율적 → 동적 계획 전략 적용
  - 예) (10, 60), (20, 100), (30, 120)인 세 물건 A, B, C

넣는 물건	A	B	C	A, B	B, C	A, C	A, B, C
무게 합	10	20	30	30	50	40	60
가치 합	60	100	120	160	220	180	280

0-1배낭채우기 알고리즘 (억지기법)



- 완전 탐색 : 가능한 모든 경우 만들자
  - 원소가 n개일때 조합의 수 : 2^n개 => $O(2^n)$

```
01: def Knapsack01_BF(wgt, val, W):
02:     n = len(wgt)            # 전체 물건의 수
03:     bestVal = 0             # 배낭의 최대 가치
04:
05:     for i in range(2**n) :
06:         s = [0]*n
07:         for d in range(n) :
08:             s[d] = i%2
09:             i = i//2
10:
11:         sumval = 0
12:         sumWgt = 0
13:         for d in range(n):
14:             if s[d] == 1 :
15:                 sumWgt += wgt[d]
16:                 sumVal += val[d]
17:
18:         if sumWgt <= W :
19:             if sumVal > bestVal :
20:                 bestVal = sumVal
21:
22:     return bestVal  # 최대 가치 반환
```

부분집합의 수는 2^n이므로, i에 0부터 2^n-1까지를 순서대로 대입함.

i를 이진수로 변환했을 때, 각 자리의 수를 리스트에 저장(역순으로). 예를 들어, 6은 이진수로 110인데, [0, 0, 1]로 저장함. 리스트에서 1이 포함되는 물건을 가리키도록 함.

현재 경우(i)에 대한 물건의 총 무게와 총 가치를 구함.

가능한 부분 집합이고, 가치합이 최대 가치보다 크면 최대 가치 갱신

```
weight = [10,  20, 30]  # 물건별 무게
value  = [60, 100, 120] # 물건별 가치
W = 50
print("억지기법:", Knapsack01_BF(weight, value, W))
```

```
wgt = [10, 20, 30]    # 물건별 무게
val  = [60, 100, 120] # 물건별 가치
W = 50                # 배낭의 제한 용량
```

이진수 변환 루프

예) **n이 3**이면 0~7의 **8가지 경우** 발생

0은 000로 하나도 안 넣는 경우

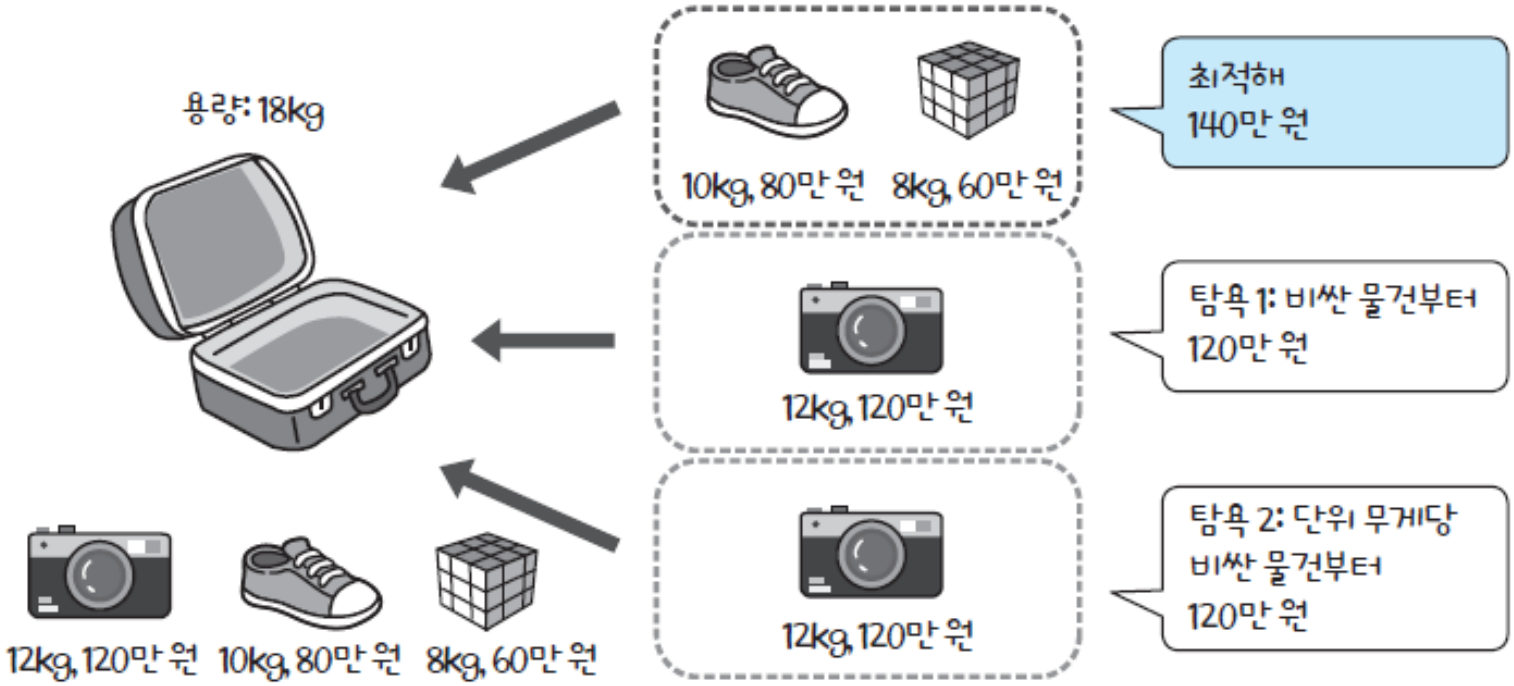
7은 111로 모두 넣는 경우

0	1	2	4	3	6	5	7
[0,0,0]	[1,0,0]	[0,1,0]	[0,0,1]	[1,1,0]	[0,1,1]	[1,0,1]	[1, 1, 1]
넣는 물건	A	B	C	A,B	B,C	A,C	A,B,C
무게 합	10	20	30	30	50	40	60
가치 합	60	100	120	180	220	180	280

```
04: bestLst = []
19:     if sumVal > bestVal :
                bestVal = sumVal
                bestLst = list(s)
21: print(bestVal, bestLst)
#220, [0, 1, 1]
```

분할가능한 배낭채우기 (탐욕적기법)

- 0-1 배낭채우기 문제는 탐욕적 기법으로 최적해를 구하지 못함



- 분할가능한 배낭채우기만 최적해를 구함

분할가능한 배낭채우기 알고리즘 (탐욕적기법)

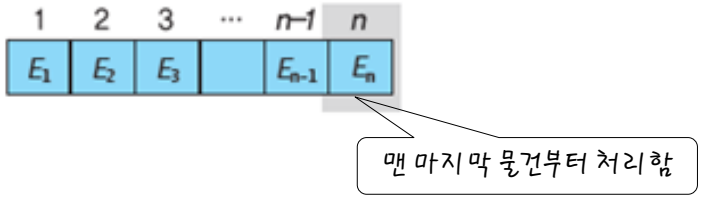
```
# 분할 가능한 배낭 채우기(탐욕적 기법)
def KnapSackFrac(wgt, val, W):
    bestVal = 0                                 # 전체 배낭의 가치
    for i in range(len(wgt)):
        if W <= 0 :                             # 용량이 다 찬 경우
            break
        if W >= wgt[i]:                         # 물건 전체를 넣을 수 있으면 넣고
            W -= wgt[i]                         # 남은 용량 W갱신
            bestVal += val[i]                   # 최대가치 증가
        else:                                   # 물건의 일부만 넣을 수 있으면
            fraction = W / wgt[i]               # 비율 계산
            bestVal += val[i] * fraction        # 최대가치가 비율만큼 증가
            break                               # 종료
    return bestVal

# 테스트
weight = [30,  20, 10]          # 물건별 무게 (가치/무게 의 내림차순 정렬)
value  = [120, 100, 60]         # 물건별 가치 (가치/무게 의 내림차순 정렬)
W = 50                          # 배낭의 제한 용량
print("분할가능한 배낭채우기 :", KnapSackFrac(weight, value, W))
```

O(n)

0-1배낭채우기 문제의 순환 관계식

- 배낭 문제의 순환 관계식 만들기
  - 배낭 용량 W
  - n개의 물건: E_1, E_2, …, E_n
  - $E_i = (wt_i, val_i)$
  - $A(n, W)$: 용량이 W인 배낭에 n개의 물건을 넣을 때의 최대 가치



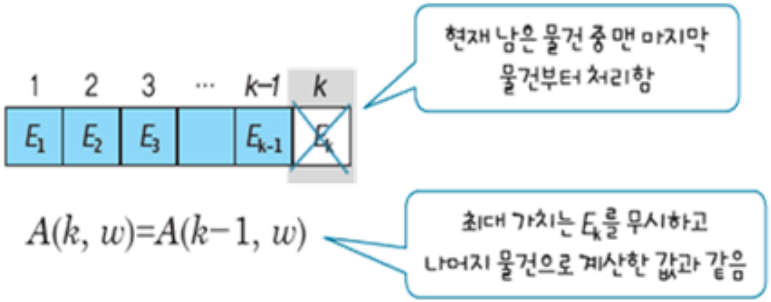
- $case\ 1:\ A(k, w)$의 기반 상황
  - $A(0, w) = 0,\quad A(k, 0) = 0$: 넣을 물건이 없거나, 남은 용량이 0인 경우

k : 남은(넣을) 물건이 없음

w : 남은 용량이 0

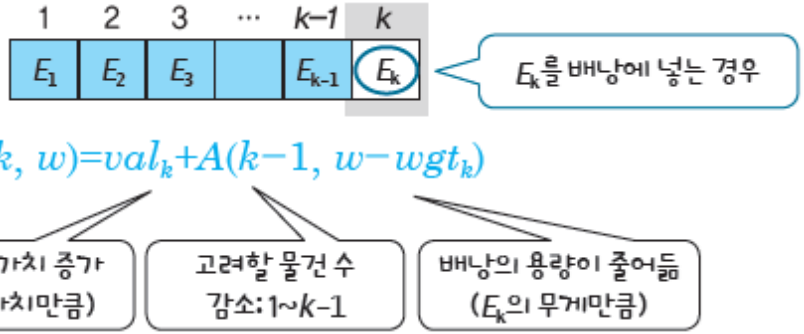
- $case\ 2:\ A(k, w)$의 일반 상황
  - $wgt_k > w$: 마지막 물건이 배낭 용량보다 무거운 경우

  → 넣을 수 없음
    - $A(k, w) = A(k-1, w)$

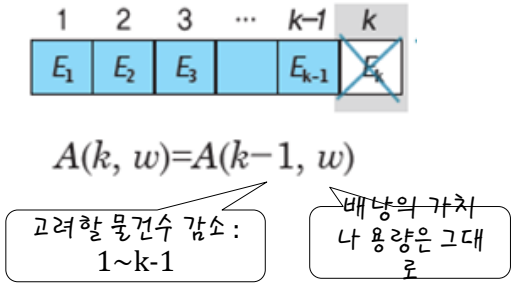


0-1배낭채우기 문제의 순환 관계식

- $case\ 3 : A(k,w)$의 일반 상황
  - $wt_k \le w$: 넣거나, 안 넣거나
    - 넣은 경우 : $val_k + A(k-1, w-wt_k)$



- 넣지 않은 경우 : $A(k-1, w)$

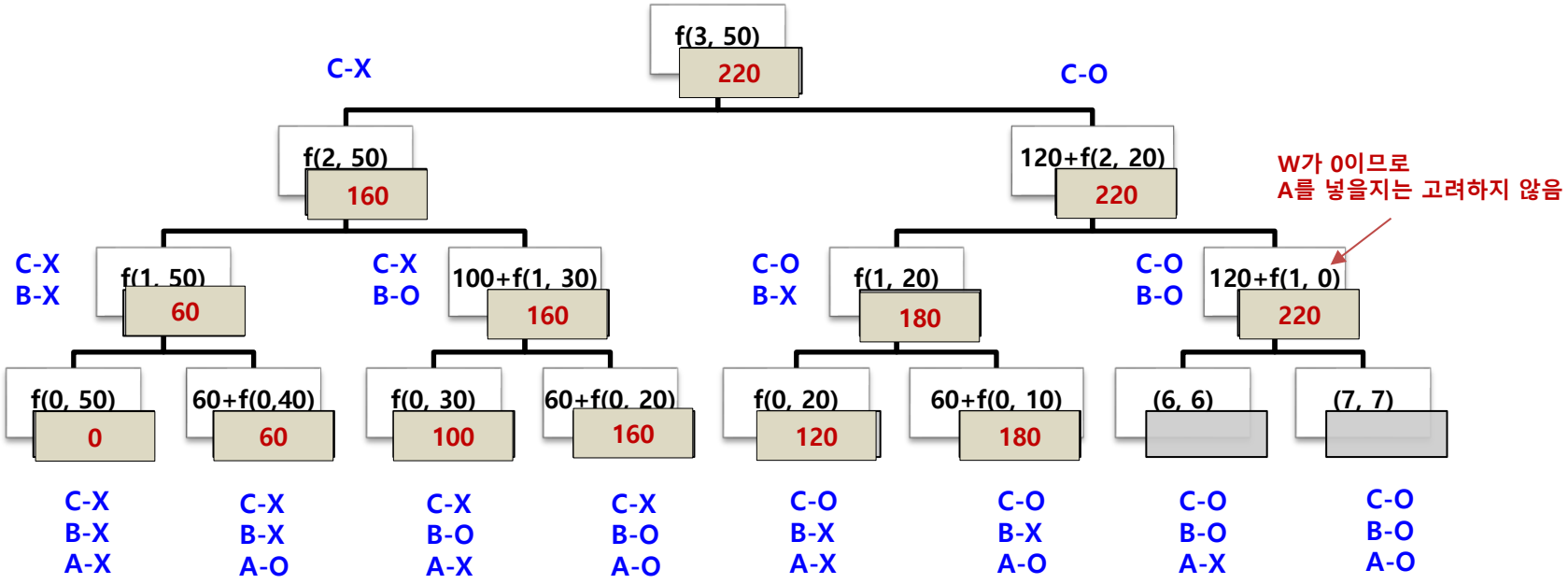


$$A(n,W) = \begin{cases} 0 & \text{if } W=0 \text{ or } n=0 \\ A(n-1, W) & \text{if } wt_n > W \\ \max(val_n + A(n-1, W-wt_n),\ A(n-1, W)) & \text{otherwise} \end{cases}$$

0-1배낭채우기 알고리즘 (분할정복기법)

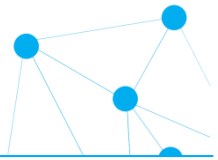
넣는 물건	A	B	C	A, B	B, C	A, C	A, B, C
무게 합	10	20	30	30	50	40	60
가치 합	60	100	120	160	220	180	280

```
weight = [10,  20, 30]  # 물건별 무게
value  = [60, 100, 120] # 물건별 가치
W = 50;  n = len(value)
print("분할정복기법:", knapSack_dc(W, weight, value, n))
```



- **valWhitout** : 왼쪽(넣지 않는 경우)
- **valWith** : 오른쪽(넣는 경우)

0-1배낭채우기 알고리즘 (분할정복기법)



```
# 0-1 배낭 채우기(분할정복기법)
def knapSack_dc(W, wt, val, n):
    if n == 0 or W == 0 :       # case 1 : 기반 상황
        return 0
    if (wt[n-1] > W):           # case 2 : 마지막 물건을 넣을 용량이 부족한 경우
        return knapSack_dc(W, wt, val, n-1)  # 나머지 항목으로 최대가치 구함
    else:                       # case 3 : 넣거나 넣지 않거나
        valWithout = knapSack_dc(W, wt, val, n-1)   # case3.1 : 넣지 않는 경우
        valWith = val[n-1] + knapSack_dc(W-wt[n-1], wt, val, n-1) # case3.2 : 넣는 경우
        return max(valWith, valWithout)              # 둘 중 더 큰 값

weight = [10,  20, 30]      # 물건별 무게
value  = [60, 100, 120]     # 물건별 가치
W = 50                      # 현재 배낭 용량
n = len(value)              # 물건수
print("분할정복기법:", knapSack_dc(W, weight, value, n))
```

- n=10 → 1,024회 연산
  n=30 → 약 10억회 연산
  n=100 → 우주의 나이보다 오래 연산
- **시간복잡도 *O(2ⁿ)* 비효율적 알고리즘 → 동적계획법을 통해 개선**
- 물건의 무게나 배낭의 용량을 실수로 할 경우 다른 알고리즘을 적용하기 어려움

0-1배낭채우기 (동적계획법 : 테이블화)

- 적용 조건
  - 물건의 무게와 배낭의 용량을 모두 정수로 제한

- 테이블 설계
  - 함수 *A(n, W)*
  - 2차원 테이블 : (n+1) * (W+1)
  - 기반 상황 : A(0, W)=0, A(n, 0)=0 물건이 없거나(n=0), 배낭용량이 0(W=0)인 경우
    → 최대가치는 0 (회색음영)

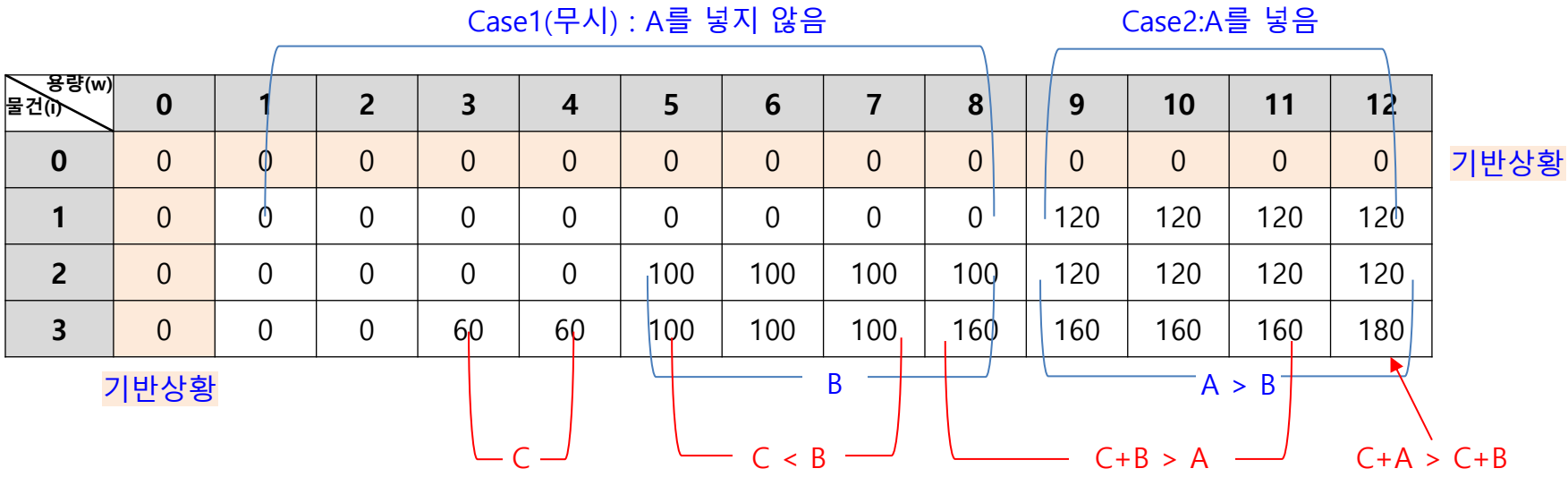
		배낭 용량							
		0	1	…	w−wt_i	…	w	…	W
물건	0	0	0	…	0	…	0	…	0
	1	0							
	…	…							
	i−1	0			$A(i-1, w-wt_i)$	…	$A(i-1, w)$		
	i	0					$A(i, w)$		
	…	…							
	n	0							$A(n, W)$

0-1배낭채우기 (동적계획법:테이블화)

- 적용 조건 : 배낭의 용량 12

	A	B	C
물건의 무게	9	5	3
물건의 가치	120	100	60

- 테이블 : 최대가치 저장



0-1배낭채우기 알고리즘 (동적계획법:테이블화)

```
# 0-1 배낭 채우기(동적 계획법:테이블화)
def knapSack_dp(W, wt, val, n):
    A = [[0 for x in range(W + 1)] for x in range(n + 1)]

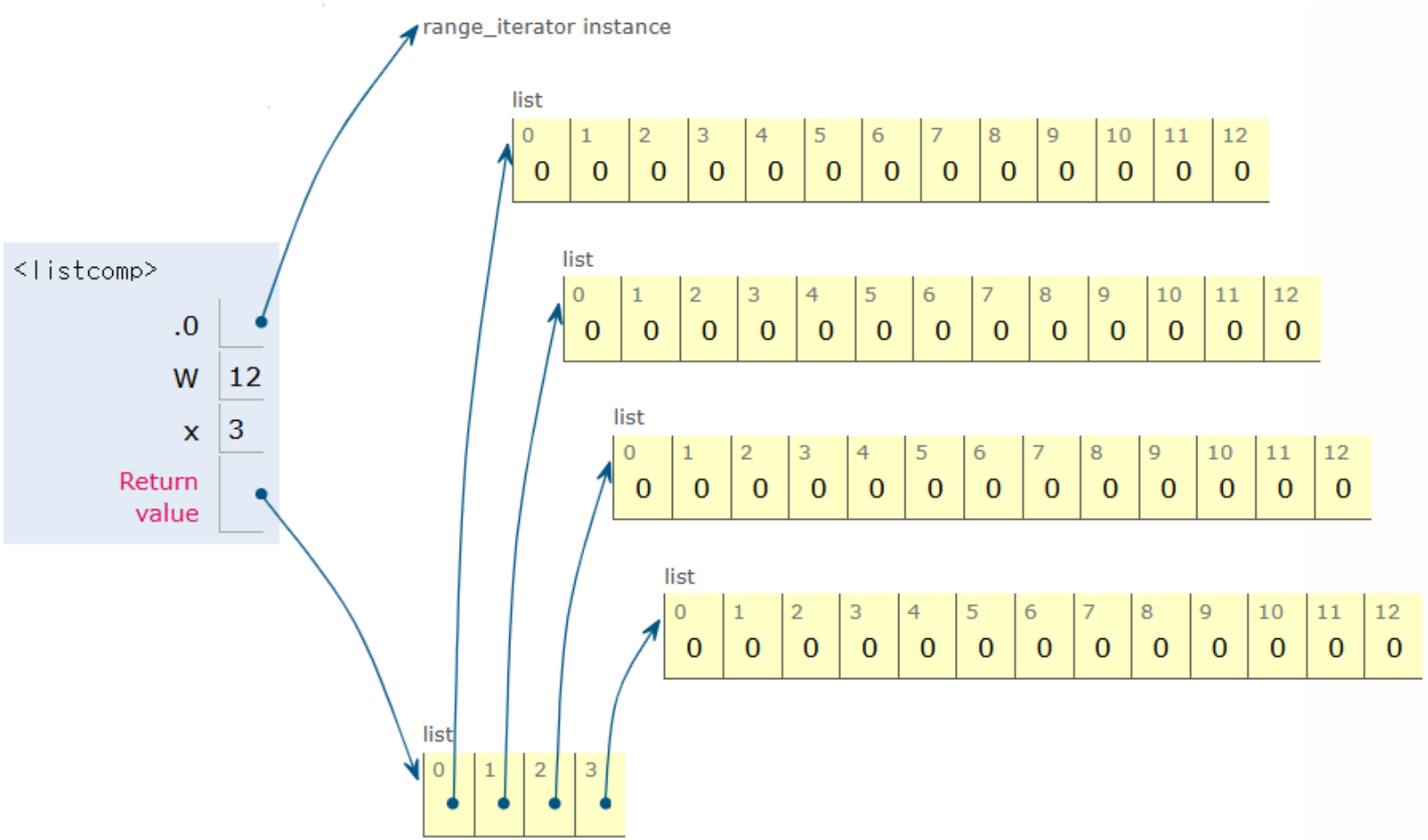
    for i in range(1, n + 1):                          # n번(물건수)
        for w in range(1, W + 1):                      # W번(배낭용량만큼)
            if wt[i-1] > w:                            # case1: i번째 물건이 배낭 용량 초과
                A[i][w] = A[i-1][w]     # i번째 물건을 무시하고 나머지로 계산(계산되어 있음)
            else :                                     # case2: i번째 물건이 배낭 용량 이하
                valWith = val[i-1] + A[i-1][w-wt[i-1]]  # case2.1: i번째 물건을 넣는 경우
                valWithout = A[i-1][w]                  # case2.2: i번째 물건을 넣지 않는 경우
                A[i][w] = max(valWith, valWithout)     # 더 큰 값 선택
        print(A[i])
    return A[n][W]       # 최종 결과

weight = [9,  5,  3]       # 물건별 무게
value  = [120, 100, 60]  # 물건별 가치
W = 12
n = len(value)
print("0-1배낭문제(동적 계획): ", knapSack_dp(W, weight, value, n))
```

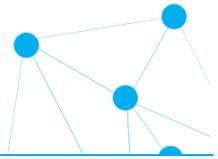
```
[0, 0, 0, 0, 0, 0, 0, 0, 0, 120, 120, 120, 120]
[0, 0, 0, 0, 0, 100, 100, 100, 100, 120, 120, 120, 120]
[0, 0, 0, 60, 60, 100, 100, 100, 160, 160, 160, 160, 180]
0-1배낭문제(동적 계획):  180
```

0-1배낭채우기 알고리즘 (동적계획법:테이블화)

```
A = [[0 for x in range(W + 1)] for x in range(n + 1)]
```



0-1배낭채우기 (동적계획법)



- 복잡도 분석
  - 시간 복잡도 / 공간 복잡도 *: O(nW)*

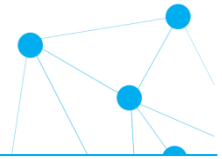
    억지기법 $O(2^n)$ 이나 분할정복$O(2^n)$보다 월등
  - 배낭의 용량이 매우 크거나 물건의 종류가 많다면 추가 공간과 처리시간이 모두 크게 늘어남
  - 배낭 용량이나 물건의 개수가 적을때는 분할정복이 유리

0-1배낭채우기 알고리즘 (동적계획법:메모이제이션)

```
#0-1 배낭채우기 (동적계획법:메모이제이션)
def knapSack_mem(W, wt, val, n):
    if A[n][W] == None :
        if n == 0 or W == 0 :
            A[n][W] = 0
        elif (wt[n-1] > W):
            A[n][W] = knapSack_mem(W, wt, val, n-1)
        else:
            valWithout = knapSack_mem(W, wt, val, n-1)
            valWith = val[n-1] + knapSack_mem(W-wt[n-1], wt, val, n-1)
            A[n][W] = max(valWith, valWithout)
    return A[n][W]

weight = [9,  5,  3]        # 물건별 무게
value  = [120, 100, 60]     # 물건별 가치
W = 12
n = len(value)
A = [[None] * (W+1) for _ in range(n+1)]
print(A)
print("0-1배낭문제(동적 계획:메모이제이션): ", knapSack_mem(W, weight, value, n))
```

연습문제

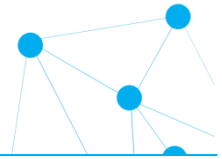


1. 동적계획법을 적용하여 0-1배낭채우기 문제를 해결하시오. 배낭의 용량은 6일때 테이블을 만들고 모든 항목을 채워 답을 구하는 과정을 보이시오.

	물건1	물건2	물건3	물건4	물건5
무게	3	2	1	4	5
가치	26	20	14	40	50

용량 / 물건	0	1	2	3	4	5	6
	0	0	0	0	0	0	0
물건1	0						
물건2	0						
물건3	0						
물건4	0						
물건5	0						

연습문제



2. 0-1 배낭 채우기의 동적계획법 알고리즘에 대해 물음에 답하시오.

1) 그리디 알고리즘 기법은 분할 가능 배낭문제는 최적해를 구하지만 0-1 배낭문제는 최적해를 보장하지 못하는데 그 이유는?

2) 배낭의 최대 용량이 W이고, 물건이 개수가 N일때 시간 복잡도는?

3) 무게가 각각 3kg, 4kg, 5kg이고, 가치가 40, 50, 60인 3개의 물건이 있다. 최대 7kg 용량의 배낭이 있을때, 동적계획법으로 배낭의 최대 가치는?

4) DP테이블 K[i][w]를 채울때, 현재 검사중인 i번째 물건의 무게 Wi가 현재 배낭의 남은 용량 W보다 커서 배낭에 넣을 수 없을 경우, K[i][w]에 대입해야 하는 우변의 값은?

5) K[i][W] = max(K[i-1][W], val[i-1]+K[i-1][W-wt[i-1]])에서 밑줄친 val[i-1]+K[i-1][W-wt[i-1]]이 의미하는 것은?



감사합니다!